

# Concourse: Amendments to the Queueing Model Definition

Concourse research notes

July 17, 2026

## Contents

|          |                                                                           |          |
|----------|---------------------------------------------------------------------------|----------|
| <b>1</b> | <b>What this document is</b>                                              | <b>1</b> |
| <b>2</b> | <b>A1: clock bookkeeping is model state</b>                               | <b>2</b> |
| 2.1      | The argument . . . . .                                                    | 2        |
| 2.2      | The amended state . . . . .                                               | 3        |
| 2.3      | Contract delta CD-1: <code>fire</code> receives the firing time . . . . . | 4        |
| <b>3</b> | <b>A2: the clock-family set is open</b>                                   | <b>4</b> |
| 3.1      | The refutation of “two families suffice” . . . . .                        | 4        |
| 3.2      | The family registry . . . . .                                             | 4        |
| <b>4</b> | <b>A3: every function-valued field is a combinator value</b>              | <b>5</b> |
| 4.1      | The argument . . . . .                                                    | 5        |
| 4.2      | The scalar-expression algebra . . . . .                                   | 5        |
| 4.3      | The escape hatch . . . . .                                                | 5        |
| <b>5</b> | <b>A4: no hidden randomness</b>                                           | <b>6</b> |
| <b>6</b> | <b>A5: state-dependent service laws carry age across segments</b>         | <b>6</b> |
| <b>7</b> | <b>The amended contract surface</b>                                       | <b>7</b> |
| <b>8</b> | <b>The falsification charter</b>                                          | <b>8</b> |
| <b>9</b> | <b>Queued middle-layer questions</b>                                      | <b>8</b> |

## 1 What this document is

`queue_layers.tex` (in this directory) defines a three-layer queueing library: a surface language, a GSMP intermediate representation, and the `CompetingClocks` sampler. `Concourse.jl` adopts that design with the following changes, and this note is the normative statement of them. Where the two documents disagree, this one wins; everything not amended here carries over unchanged.

The amendments, agreed in design discussion on July 17, 2026:

- A1.** *Clock bookkeeping is model state.* Per-clock enabling times and banked ages move into `QueueState`. The sampler receives them; it does not own them. (Section 2.)

- A2.** *The clock-family set is open.* `queue_layers.tex` claimed two clock families suffice; reneging and server breakdowns refute that as a design ceiling. Families become a registry the interpreter iterates, not a closed enumeration it hard-codes. (Section 3.)
- A3.** *Every function-valued field is a combinator value.* Laws, mark samplers, discipline components, and routing kernels are inspectable expression values with derivable reads, with one explicitly-marked opaque escape hatch. (Section 4.)
- A4.** *No hidden `rand()`.* Every auxiliary draw — marks, probabilistic routing, SIRO’s pick — consumes an explicit, keyed, recorded draw source that appears in the function’s signature. (Section 5.)
- A5.** *State-dependent service laws carry age across segments.* A station’s service law may read live occupancy; when a watched count changes, each surviving clock is re-declared with the fresh law conditioned on its accrued service effort, `te` fixed. (Added July 18, 2026; Section 6.)

One contract delta against `ClockGradients.jl` falls out of A1 and is proposed in Section 2.3: an optional four-argument `fire(model, state, key, t)`.

A second decision frames the whole note: `Concourse` depends on `CompetingClocks.jl` and `ClockGradients.jl` and *not* on `ChronoSim.jl`. This is cheaper than `queue_layers.tex` assumed, because every seam it borrowed from `ChronoSim` exists in the two remaining packages: `:resume` memory is `enable!` with a left-shifted enabling time (`te < when` is documented behavior of the sampler interface); processor sharing’s mid-flight re-evaluation is `reenable!`; the record enters the estimators through the `gradient_record` generic, which has no core methods precisely so that frameworks can attach their own. The one substantial replacement is the nine-verb branchable-world protocol for the clone-based estimators, deferred to phase 2. `Concourse` thereby becomes the second independent implementation of the `ClockGradients` model contract, beside the `ChronoSim` package extension — which is the test that the contract is an abstraction and not a description of its only client.

**Falsification stance.** Following the practice in `queue_layers.tex` §5, every design claim in this note appears in Section 8 paired with the concrete phase-1 test that would falsify it. The phase-1 prototype exists to run that table, not to be a library.

## 2 A1: clock bookkeeping is model state

### 2.1 The argument

A GSMP’s Markov state is  $(s, \text{clock readings})$  — Glasserman Ch. 2 gives the canonical construction. `queue_layers.tex` put  $s$  in `QueueState` and left the readings in the sampler. That split fails twice:

1. *It breaks `fire`’s purity for age-reading disciplines.* SRPT’s `preempt` compares remaining sizes, which are size marks minus ages; `:resume` re-enabling needs the banked age. If ages live only in the sampler, `settle!` must call `enabled_ages` on the sampler mid-fire, which inverts the layering and makes `fire` depend on an argument it does not receive.

2. It makes the four-argument *clock\_distribution* ambiguous under replay. ClockGradients' score replay evaluates `clock_distribution(model,  $\theta$ , k, state)` for every enabled clock  $k$  at every inter-event interval — at states *later* than  $k$ 's enabling (`src/score.jl:53-58`). A time-varying law (nonhomogeneous arrivals, a shift schedule) must construct its distribution anchored at  $k$ 's enabling time  $t_e$ . If the model reads a single “current time” field, a long-enabled clock gets the wrong anchor at later states. Only a *per-clock*  $t_e$  table inside the state gives the same answer at every state from which the replay asks.

The replay machinery already concedes the point: `score_loglikelihood` maintains an external per-clock enabling-time table via `sync_enabling_times!` because the likelihood cannot be computed without one. A1 moves that table to where the formalism says it lives.

## 2.2 The amended state

```

struct QueueState
  time::Float64           # time of the firing that produced this state
  buf::Vector{Vector{JobId}} # per station, in discipline order
  srv::Vector{Vector{JobId}} # per station, the jobs in service
  jobs::Dict{JobId,Job}     # marks; birth time lives in the record
  pending::Vector{Dict{GroupId,Vector{JobId}}} # joins: siblings so far
  cursor::Vector{Int}       # stateful routing kernels
  next_id::JobId
  te::Dict{ClockKey,Float64} # enabling time, per enabled clock
  bank::Dict{ClockKey,Float64} # accrued age, per disabled :resume clock
end

```

Bookkeeping rules, all inside `fire/settle!`:

- A clock entering the enabled set at firing time  $t$  gets `te[k] = t - bank[k]` (with `bank[k] = 0` when absent), and its bank entry is deleted. The left-shifted  $t_e$  is exactly what `enable!(sampler, k, dist, te, when)` receives, so the sampler and the state agree by construction.
- A clock leaving the enabled set without firing, under a discipline with `memory = :resume`, banks its age: `bank[k] += t - te[k]`. Under `:fresh` the bank entry is deleted. Firing deletes both entries.
- A clock's age at any state is `st.time - st.te[k]` for enabled clocks and `st.bank[k]` for disabled ones. SRPT's `preempt` and the PS re-enabling read *these*; no model code calls `enabled_ages` on the sampler, ever. The sampler is a pure consumer of state-derived arguments.

Two consequences worth naming. First, `states_equal` and record-fold replay now compare and reproduce the clock bookkeeping too, so a divergence between the model's ages and the sampler's is detectable rather than silent — the runtime invariant is `st.te == sampler's enabling times` after every firing. Second, the state stays plain data: two Dicts of `isbits` pairs copy, hash, and diff like everything else in it.

## 2.3 Contract delta CD-1: fire receives the firing time

`fire` must stamp `st.time` and the new `te` entries, so it needs the firing time — which the current `ClockGradients` contract does not pass: `fire(model, state, key)` is called at twelve sites across `score.jl`, `spa.jl`, `functionals.jl`, `clockworld.jl`, `records.jl`, and `conformance.jl`, and at every site the firing (or swap) time is already in scope in the caller. The proposal:

```
# In ClockGradients: the time-aware form, with a fallback so existing
# time-free models are untouched.
fire(model, state, key, t) = fire(model, state, key)
# All internal call sites switch to the four-argument form.
```

Backwards compatible, mechanical, and confined to the user’s own package. Concourse implements only the four-argument method. The alternative — keeping `fire` time-free and expressing every law in age coordinates — was rejected because it silently forbids wall-clock-inhomogeneous laws, which are a stated goal (“time-varying hazards for rates”).

SPA’s hypothetical firings (`spa.jl:70,99,271`) pass the swap time; that is the semantically correct anchor for the alternate world’s new enablings, and the criticality gate’s state comparisons inherit it.

## 3 A2: the clock-family set is open

### 3.1 The refutation of “two families suffice”

`queue_layers.tex` finding 1 claimed arrival and service clocks cover the design. The claim holds for its chosen feature set and fails one step outside it: *reneging* — a patience clock on a job that is waiting, not in service — is bread-and-butter queueing ( $M/M/1+M$ , every call-center model) and is a third family. Server breakdowns and vacations are a fourth. The finding is therefore demoted from an architectural assumption to a statement about the initial feature set, and the architecture must make adding a family cheap.

### 3.2 The family registry

`ClockKey` stays one concrete isbits tuple; the `Symbol` field is the open dimension:

```
const ClockKey = Tuple{Symbol,Int32,Int64}
# (:arrival, station, 0)      the source’s next-arrival clock
# (:service, station, jobid) one clock per job in service
# (:patience, station, jobid) one clock per waiting job, if reneging [later]
# (:repair, station, srvidx) one clock per down server, if breakdowns [later]
```

A clock family is a value with three functions — the same move A3 makes for laws, applied to the interpreter itself:

```
struct ClockFamily
  name::Symbol
  enumerate # (model, state) -> iterator of ClockKey (who is enabled)
  law       # (model, θ, key, state) -> UnivariateDistribution
  fire      # (model, state, key, t) -> new state (what firing means)
end
```

---

`QueueGSMP` carries `families::VectorClockFamily`; `enabled` is the concatenation of each family’s `enumerate` in registry order (deterministic order, as the contract requires); `clock_distribution` and `fire` dispatch on `key[1]` through the registry. The interpreter, the record fold, the checker’s randomness census, and the estimators never mention a family by name. The acid test is F2 in the charter: adding `:patience` must touch the registry and nothing else.

What stays closed: a family’s `fire` still works through the shared `deposit!/settle!` vocabulary, so disciplines and routing compose with new families instead of each family reinventing station semantics. (A patience firing is “remove job  $j$  from `buf`, deposit to the reneging destination”; a repair firing is “increment station capacity, `settle!`”.)

## 4 A3: every function-valued field is a combinator value

### 4.1 The argument

The IR’s advertised virtue is that it is *data* — printable, diffable, hashable, checkable. `CompiledStation`’s `service::Function`, `routing::Function`, `mark::Function` fields quietly forfeit that: closures cannot be hashed, compared, serialized, or introspected, and the checks that justify the IR — C2 (mark typing) and C6 (randomness census / estimator tiers) — need to know what a law reads and whether it depends on  $\theta$ . The routing kernels already show the fix (`Always`, `ByMark`, `JoinShortestQueue` are values); A3 applies it uniformly.

### 4.2 The scalar-expression algebra

One small expression type feeds every parameter slot:

```
# ScalarExpr: a tree with derivable reads.
Param(:mu)          # reads  $\theta$  by declared name ->  $\theta$ -dependence is visible
Mark(:size)          # reads a job mark           -> C2 dataflow is visible
Enab()               # the wall-clock enabling time  $t_e$ 
Const(2.0)
+, *, inv, log, ... # lifted arithmetic on ScalarExpr
```

A *law* is a distribution family applied to scalar expressions; a *mark sampler* is a named tuple of distribution expressions; a *discipline* is the four-field value of `queue_layers.tex` §2.2 with its by-style components as `ScalarExpr` rather than closures:

```
service = Law(Exponential, scale = Mark(:size) * inv(Param(:mu_short)))
mark     = MarkLaw(size = Law(LogNormal, = Const(0.0), = Const(1.0)))
disc     = Priority(by = Mark(:class); preempt = true, memory = :resume)
```

Derived, not declared: `reads_ $\theta$ (law)`, `reads_marks(law)`, `reads_time(law)` walk the tree. C2 becomes a dataflow check over values; C6’s census and the estimator-tier table are computed, not asserted. Nonhomogeneous laws use `Enab()`; state-reading kernels (`JoinShortestQueue`) remain the named combinators they already were.

### 4.3 The escape hatch

```
OpaqueLaw(f; reads_θ = true, reads_marks = (:size,), reads_time = false)
```

An opaque law carries *mandatory* declared reads; the checker trusts the declaration, marks every conclusion resting on it as “declared, unverified,” and degrades the estimator tiers it cannot certify. A closure with no declaration is a construction-time error, not a warning. The same wrapper exists for kernels and discipline components. The charter’s F3 measures whether the algebra is rich enough that the hatch stays shut for the standard models.

## 5 A4: no hidden randomness

The rule from `queue_layers.tex` §3.5 — every draw is a clock, a recorded mark, or a recorded choice — becomes a signature-level fact. Auxiliary draws (marks at arrival, `Probabilistic` routing, SIRO’s pick) consume an explicit draw source:

```
# The draw source is an argument wherever auxiliary randomness may occur.
select(disc, buf, state, draws)           # SIRO reads; FCFS ignores
route(kernel, job, state, draws)          # Probabilistic reads; ByMark ignores
draw_marks(marklaw, draws)                # at arrival firings
```

Two modes, one type. In live simulation, `draws` wraps CompetingClocks’ `KeyedStreams`, keyed by (`firing ClockKey`, `purpose::Symbol`) so that a draw belongs to a clock and purpose rather than to a position in a global call sequence — the same common-random-numbers discipline the sampler already imposes on clock randomness, extended to the auxiliary draws. Every value drawn is appended to the current record entry. In replay, `draws` is the recorded sequence, and drawing returns the recorded values in consumption order — which is precisely the conditioning that makes `fire` deterministic given the record, and which the likelihood’s mark and routing-mass terms require.

The marked record is therefore

$$\mathcal{R} = ((k_i, t_i, a_i))_i, \quad a_i = \text{the auxiliary draws consumed by firing } i, \text{ in order,}$$

`queue_layers.tex`’s record with the mark column generalized to “all auxiliary draws.” The checker’s census (C6) enumerates the possible draws per family from the combinator values (A3), so record-schema-vs-model agreement is itself checkable.

## 6 A5: state-dependent service laws carry age across segments

Added July 18, 2026, with the state-dependent-service capability; the phase-1 amendments above are unchanged by it.

A station’s service law may read live occupancy through two new atoms of the A3 algebra, `InService(station)` and `InBuffer(station)`, with `reads_state` joining the derived censuses. Occupancy is live state, not a value frozen at enabling, so the read is scoped: only the service law of a *station* may read occupancy. State in a mark law would push station state into the record’s mark draws, violating A4’s conditioning; a state-reading interarrival law is balking, a feature of its own; patience laws, routing kernels, and discipline ordering keys are refused likewise. This is the compile check the code registry numbers C5 (see the registry remark below).

The semantics is the *age-carryover segment convention*. Compile builds a dependency map (`srv_readers/buf_readers`) from the census. When a firing changes a watched in-service or buffer count at time  $t$ , every surviving in-service clock of a reader station closes a segment: it banks the effort accrued since its last anchor at the old speed ( $\min(1, \text{servers}/n)$  under PS, 1 elsewhere), moves its anchor to  $t$ , keeps `te` at the original enabling — `te` is never rewritten — and is re-declared to the sampler as `(:reenable, k)`. The re-declared law is the *fresh base*  $F_{\text{new}}$ , rebuilt from the occupancy at  $t$ , conditioned on the total draw exceeding the accrued effort  $a$ :  $X \sim F_{\text{new}} \mid X > a$ , delivered as a wall-time distribution anchored at  $t$  (the existing `SharedRemaining` wrapper). Processor sharing becomes a special case of this general path: a PS station is an `srv-reader` of itself, with base unchanged and speed  $\min(1, \text{servers}/n)$ .

The rejected alternative was the quantile-equivalent (hazard-matching) mapping, which carries the old law’s survival quantile into the new law so that survival probability is continuous across the change. Age carryover won on two grounds: it is what processor sharing already implements, and it is what the `ClockGradients` fixed-anchor contract expects — `te` anchors every likelihood segment, and the score replay rebuilds the current segment’s law at every state it visits. Pathwise branching is *not* admitted for these models (`branch_world` refuses): reordering events changes an occupancy, hence the law, discontinuously, and the clone estimators’ unbiasedness argument does not cover that coupling. The score estimator remains valid; the stability check (C4) skips state-reading stations rather than invent a static mean.

**Registry remark.** `queue_layers.tex` §3.7 numbers its checks C1–C6 with C5 = oracle detection and C6 = randomness census, and this note cites those meanings above (Sections 4, 5, 8). Concourse’s *compile-time* registry diverged: its C5 is the state-reading scope check of this amendment (the oracle detector is not a compile check in Concourse). In this note, “compile check C5 (state-reading scope)” refers to the code’s registry; an unqualified C5 elsewhere in this document keeps the `queue_layers.tex` meaning.

## 7 The amended contract surface

What Concourse implements, in `ClockGradients`’ vocabulary. Deltas against the shipped contract are marked.

| Contract function                                              | Concourse implementation                                          | Delta     |
|----------------------------------------------------------------|-------------------------------------------------------------------|-----------|
| <code>initial_state(m)</code>                                  | empty places; <code>te</code> , bank empty; <code>time = 0</code> | —         |
| <code>clockkeytype(m)</code>                                   | <code>Tuple{Symbol, Int32, Int64}</code>                          | —         |
| <code>enabled(m, st)</code>                                    | concat of family <code>enumerates</code> , registry order         | —         |
| <code>clock_distribution(m, <math>\theta</math>, k, st)</code> | family law; anchors from <code>st.te</code>                       | —         |
| <code>fire(m, st, k, t)</code>                                 | family fire through <code>deposit! / settle!</code>               | CD-1      |
| <code>gradient_record(m, rec, <math>\theta</math>)</code>      | one method for the marked record type                             | by design |

Replay determinism is conditional on the recorded draws (A4), exactly as the `ChronoSim` extension conditions on its recorded firings. The nine-verb branchable world over (`QueueGSMP`, `QueueState`, `SamplingContext`) is phase 2; `check_branchable` is its acceptance test.

## 8 The falsification charter

Each claim is stated so that the phase-1 (or explicitly phase-2) prototype can refute it. A failed test is a design discovery, not a bug to paper over. Oracles follow the 4-standard-error convention; closed forms come from the C5 detector cases (M/M/1, the SITA farm as two independent M/G/1 queues, Erlang-A for reneging).

- F1 (A1 sufficiency).** With `te/bank` in state, no model code needs sampler introspection. *Test:* SRPT and preempt-:resume runs match their oracles while the model layer has no reference to `enabled_ages`; the `st.te-vs-sampler` invariant holds after every firing of a long mixed run.
- F2 (A2 openness).** A new clock family costs one registry entry. *Test:* implement `:patience` (reneging); diff the change — if any file other than the family registry and the surface constructor changes, the claim is falsified. M/M/1+M against Erlang-A.
- F3 (A3 coverage).** The scalar algebra covers the five `queue_layers.tex` models plus reneging with zero `Opaque` escapes. *Test:* build all six; count escapes.
- F4 (A4 determinism).** `fire` is deterministic given the record. *Test:* fold the marked record and compare `states_equal` against the live trajectory at every index, including `te` and `bank`.
- F5 (CD-1 sufficiency).** The four-argument `fire` is the only ClockGradients change queueing needs. *Test:* score and IPA on a nonhomogeneous-arrival M(t)/M/1 against finite differences, running on patched ClockGradients with no other core edits.
- F6 (cascade totality).** Under check C3 (acyclic blocking), `deposit!/settle!` cascades terminate deterministically. *Test:* property test over random finite-buffer networks with `:block`; assert termination and replay-order equality. (The worklist order itself is a middle-layer decision, Section 9.)
- F7 (initial family set).** `{:arrival, :service}` suffices for the five `queue_layers.tex` models — the surviving, weakened form of the old finding 1. *Test:* phase 1 builds all five with two families; reneging (F2) is the designed refutation lying one step beyond.
- F8 (truncate-and-rescale, phase 2).** The speed-free compilation of PS equals an explicit-speed reference. *Test:* M/M/1-PS against its closed form, and against a Bogliolo-style clock-and-speed-matrix reference implementation on a small case.

## 9 Queued middle-layer questions

Decisions that belong to the event-loop design conversation, deliberately not made here. A1/CD-1 already settled “who owns time” (the state records it; the interpreter advances it; the sampler is told it). *Update, later the same day:* items 1, 3, and 4 are decided and item 2 is framed with a proposal in `event_loop.tex` (this directory), which is now the normative middle-layer document.

1. *Diff vs. push clock lifecycle.* Recompute `enabled` and diff after each firing (simple,  $O(\text{clocks})$  per event), or have `fire` emit its enable/disable deltas (`fire_changes` exists in the contract for this)? Phase 1 should measure before optimizing.



2. *The cascade worklist.* Order and representation for multi-hop blocking chains; the determinism argument F6 must be made for whatever is chosen.
3. *Record capture.* Interpreter-owned marked record vs. a `CompetingClocks` watcher middleware with an auxiliary-draw column bolted on.
4. *Sampler spec policy.* `NextReactionMethod` default, `FirstToFireMethod(coupling=:carry)` when IPA wants the carry coupling — who chooses, and where the choice surfaces in the API.